



Matricule N°: _____ - _____ - _____.

Nom : _____.

Prénom : _____.

Programme : _____.

GIF-1003– Programmation avancée en C++

Examen final Hiver 2021

Durée: 2h50

Aucun document autorisé

Lisez l'examen en entier avant de le commencer. Attention, chaque détail compte.

- Ce questionnaire comporte 4 parties sur 15 pages (incluant une annexe).
- Veuillez répondre dans les espaces prévus sur le questionnaire.
- Soyez concis dans vos réponses, les espaces devant être suffisants pour contenir vos

Correcteur	Points	Pages												
		1	2	3	4	5	6	7	8	9	10	11	12	13
Idrissa	30								10		14	6		
Yao	15												3	12
Ismaël	16						7	9						
Ndiogou	14			4	4	6								
Samuel	8	4	4											
Jérémie	9									9				

p1	4
p2	4
p3	4
p4	4
p5	6
p6	7
p7	9
p8	10
p9	9
p10	14
p11	6
p12	3
p13	12
Total	92

Pour la correction

1. Principes (12 points)

Test unitaire (4 points)

Quel est l'objectif des tests des cas valides?

- Vérifier la fonctionnalité des méthodes selon le cahier des charges (2)

Quel est l'objectif des tests des cas invalides?

- Vérifier la fonctionnalité des outils pour le contrat spécifiquement s'ils détectent bien les erreurs de préconditions (2)

Hérarchie de classe, Héritage et polymorphisme (4 points)

Soit la hiérarchie de classe suivante:

```
#include <string>
```

```
class Base
{
public:
    Base(std::string p_base);
    virtual ~Base(){};
private:
    std::string m_b;
};
class Derivee: public Base
{
public:
    Derivee(std::string p_base, std::string p_derive);
private:
    std::string m_d;
};
```

Pourquoi a-t-on défini un destructeur virtuel dans la classe de base, même s'il n'y a aucune ressource particulière à gérer, soit que sa définition ne contient aucune instruction? Expliquez à l'aide d'un exemple de code.

```
Base* pointerBase;
Derivee* pointerDerivee;
pointerDerivee = new Derivee("base", "derivee");
pointerBase = pointerDerivee;
delete pointerBase;
```

si le destructeur de Base n'est pas virtuel, seule la partie Base sera détruite, faisant une fuite de mémoire

En C++, par quels moyens et sous quelles conditions peut-on faire des traitements polymorphes sur un ensemble d'objets?

- avec des pointeurs (ou des références) de classe de base
- des méthodes déclarées virtuelles.

Gestion de la mémoire (4 points)

Quand est-ce qu'une classe est dite de forme canonique de Coplien ? Qu'est-ce que cela implique?

- au moins 1 des attributs renvoie à une adresse allouée dynamiquement (1)
- destructeur + constructeur copie + operator =
libération avant destruction copie en profondeur (1)

Quelles sont les conséquences des fuites de mémoire sur l'exécution d'un programme?

- Aucune si l'exécution ne demande pas beaucoup de mémoire et est très courte
- si l'exécution est longue le système finira par ne plus disposer de mémoire libre. En général le système "gèle" ou le programme "plante" (2)

Application

Nous voulons développer un programme afin de gérer un annuaire. Ce programme est construit au fur et à mesure. Sa construction est guidée par une série de questions indépendantes.

Dans tout le développement, la théorie du contrat doit être appliquée (il ne sera pas toujours précisé explicitement dans les questions qui suivent qu'il faut le faire. Alors pensez – y systématiquement.

Pour cela vous disposez des macros définies dans le fichier `ContratException.h`:

`PRECONDITION(test logique)`

`INVARIANT(test logique)`

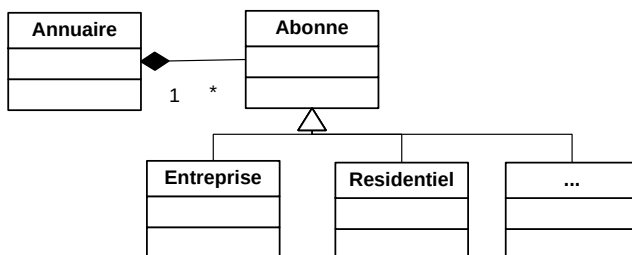
`POSTCONDITION(test logique)`

`INVARIANTS()`

`INVARIANTS()` appelle la méthode `verifieInvariant` que vous allez devoir implémenter.

2. Hiérarchie de classe, Héritage et polymorphisme

Les données manipulées correspondent à différents types d'abonné. Voici la hiérarchie de classe utilisée :



* La classe Residentiel n'est pas développée ici

Toutes les interfaces de ces classes sont données, mais ne sont pas tout à fait complètes. **Il faut les compléter au fur et à mesure si besoin.**

Ces classes seront implémentées dans l'espace de nom ***examenFinal***

Certaines méthodes sont déclarées dans ces interfaces, mais ne sont pas à implémenter, supposant qu'elles le sont déjà. Pensez à les utiliser au besoin.

N'oubliez pas d'ajouter les mots clés comme const, virtual, etc. lorsque c'est requis !

Interface de la classe Abonne (4 points):

```
// Fichier:      Abonne.h
#ifndef ABONNE_H_DEJA_INCLU
#define ABONNE_H_DEJA_INCLU

#include <string>

class Abonne
{
public:

    virtual ~Abonne() {} ;

    const std::string& reqNom() const;
    const std::string& reqTelephone() const;
    const std::string& reqAdresse() const;
    std::string reqAbonneFormate() ;

    bool operator==(const Abonne& p_abonne) const;

private:

    std::string m_nom; // ne doit pas être vide
    std::string m_telephone; // doit être validé par util::validerTelephone
    std::string m_adresse; // aucune contrainte
};

#endif // --- #ifndef ABONNE_H_DEJA_INCLU
```

4

```

1  //*****
2  // Fichier:      Abonne.h
3  //
4  // Classe:       Abonne
5  // Description:  Cette classe représente l'enregistrement d'un abonné "Abonne".
6  // Attributs:    const std::string& m_nom           : le nom de l'abonné Abonne
7  //              const std::string& m_telephone      : le numéro de téléphone du
l'abonné
8  //              const std::string& m_adresse       : l'adresse l'abonné Abonne
9  //*****
10
11 #ifndef ABONNE_H_DEJA_INCLU
12 #define ABONNE_H_DEJA_INCLU
13
14 #include "ContratException.h" // --- Les contrats
15 #include <string>
16 #include "ValidationFormat.h"
17
18 namespace examenFinal {
19
20 class Abonne
21 {
22 public:
23     Abonne(const std::string& nom, const std::string& telephone,
24           const std::string& adresse);
25
26     virtual Abonne* clone() const = 0; // virtuelle pure
27
28     const std::string& reqNom() const;
29     const std::string& reqTelephone() const;
30     const std::string& reqAdresse() const;
31     virtual std::string reqAbonneFormate() const = 0; // virtuelle pure
32
33     bool operator==(const Abonne& abonne) const;
34     virtual ~Abonne(){};
35
36 private:
37     void verifieInvariant() const;
38
39     std::string m_nom;
40     std::string m_telephone;
41     std::string m_adresse;
42 };
43
44 } // namespace examenFinal
45
46 #endif // --- #ifndef ABONNE_H_DEJA_INCLU
47

```

Interface de la classe Entreprise (6 points):

```
// Fichier:      Entreprise.h
#ifndef ENTREPRISE_H_DEJA_INCLU
#define ENTREPRISE_H_DEJA_INCLU

#include <string>

class Entreprise
{
public:

    Abonne* clone()      ;

    const std::string& reqSiteWeb() const;
    const std::string& reqTelecopie() const;

private:

    std::string m_siteWeb; //aucune contrainte
    std::string m_telecopie; // doit être validé par util::validerTelephone
};

#endif // --- #ifndef ENTREPRISE_H_DEJA_INCLU
```

```

1  //*****
2  // Fichier:      Entreprise.h
3  //
4  // Classe:       Entreprise
5  //
6  // Description: Cette classe repr sente les abonnees de la cat gorie Entreprise.
7  //                  Elle h rite la classe Abonne
8  //
9  // Attributs:   const std::string& m_siteWeb           : le site de l'entreprise
10 //                  const std::string& m_telecopie        : le num ro de t l phone du
11 //                                                            t l copieur de l'entreprise
12 //                                                            (doit  tre valid )
13 //*****
14
15 #ifndef ENTREPRISE_H_DEJA_INCLU
16 #define ENTREPRISE_H_DEJA_INCLU
17
18 #include "ContratException.h" // --- Les contrats
19 #include <string>
20 #include "ValidationFormat.h"
21 #include "Abonne.h"
22
23 namespace examenFinal
24 {
25
26 class Entreprise: public Abonne
27 {
28 public:
29     Entreprise(const std::string& nom, const std::string& telephone,
30               const std::string& adresse, const std::string& siteWeb,
31               const std::string& telecopie);
32
33     virtual Abonne* clone() const;
34
35     const std::string& reqSiteWeb() const;
36     const std::string& reqTelecopie() const;
37
38     virtual std::string reqAbonneFormate() const;
39
40 private:
41     void verifieInvariant() const;
42
43     std::string m_siteWeb; // ne doit pas  tre vide
44     std::string m_telecopie; // doit  tre valid  par util::validerTelephone
45 };
46
47 } // namespace examenFinal
48
49 #endif // --- #ifndef ENTREPRISE_H_DEJA_INCLU
50

```

Interface de la classe Annuaire (7 points):

```
// Fichier:      Annuaire.h
#ifndef ANNUAIRE_H_DEJA_INCLU
#define ANNUAIRE_H_DEJA_INCLU

#include <vector>
#include <stdexcept>
#include <string>

class Annuaire
{
public:
    Annuaire(const std::string& p_ville);

    ~Annuaire();

    std::string reqAnnuaireFormate() const;

private:

    std::string m_ville;
};

#endif // --- #ifndef ANNUAIRE_H_DEJA_INCLU
```

7


```

1  //*****
2  // Fichier:      Annuaire.h
3  //
4  // Classe:       Annuaire
5  // Description:  Cette classe repr sente un Annuaire d'une ville.
6  // Attributs:   const std::string& m_ville : le nom de Annuaire (ville)
7  //              std::vector<Abonne*> m_vAbonnes: le vecteur contenant l'annuaire
8  //*****
9
10 #ifndef ANNUAIRE_H_DEJA_INCLU
11 #define ANNUAIRE_H_DEJA_INCLU
12
13 #include <vector>
14 #include <stdexcept>
15 #include <string>
16
17 #include "Abonne.h"
18 #include "ContratException.h" // --- Les contrats
19 #include "AbonneDejaPresentException.h"
20
21 namespace examenFinal
22 {
23
24 class Annuaire
25 {
26     friend std::ostream& operator<<(std::ostream& os, const Annuaire& obj);
27 public:
28     Annuaire(const std::string& ville);
29     Annuaire(const Annuaire& p_annuaire);
30     ~Annuaire();
31
32     void ajouterAbonne(const Abonne& nouvelAbonne);
33     std::string reqAnnuaireFormate() const;
34 private:
35     const Annuaire& operator=(const Annuaire& p_annuaire) const;
36     void verifieInvariant() const;
37
38     std::vector<Abonne*> m_vAbonnes;
39     bool abonneEstDejaPresent(const Abonne& unAbonne) const;
40
41     std::string m_ville;
42 };
43 } // namespace examenFinal
44 #endif // --- #ifndef ANNUAIRE_H_DEJA_INCLU
45

```

Pour la classe `Abonne` (9 points)

On vous demande d'écrire les fonctions membre suivantes :

- un constructeur avec paramètres. Les paramètres sont le nom, le numéro de téléphone et l'adresse. Les contraintes sur les attributs sont rappelées dans l'interface (pensez au contrat). Prévoir ici l'entête de commentaires de spécification qui seront utilisés pour générer la documentation avec Doxygen (voir l'annexe pour les balises). Cet en-tête doit être complet.

7

- La fonction `reqAbonneFormate` qui doit être implantée **systematiquement** dans toute classe dérivée de la classe `Abonne`. Faites une déclaration en conséquence dans l'interface de la classe. Il n'est pas demandé de l'implémenter. Cette méthode retourne un objet string avec mise en forme ayant le format suivant :

Nom	: Blo
Adresse	: rue de la medecine
Telephone	: 418 656-2131

Pour un abonné Blo, demeurant rue de la medecine et ayant le numéro de téléphone 418 656-2131

La fonction `verifieInvariant` pour pouvoir implanter le contrat

2

Rappel : Complétez `Abonne.h` si nécessaire.

La classe `Abonne` est une classe abstraite (voir plus loin).

```

1  //*****
2  // Fichier: Abonne.cpp
3  // Classe: Abonne
4  //*****
5  #include "Abonne.h"
6  #include <sstream>
7
8  using namespace util;
9  using namespace std;
10
11 namespace examenFinal
12 {
13
14 /**
15  * \brief Constructeur qui initialise la classe avec les données fournit en argument
16  * \param[in] p_nom le nom de l'abonne
17  * \pre non vide
18  * \param[in] p_telephone le numéro de téléphone de l'abonne
19  * \pre validerTelephone
20  * \param[in] p_adresse l'adresse de l'abonne
21  * \post tous les paramètres sont assignés correctement
22  */
23 Abonne::Abonne(const std::string& p_nom, const std::string& p_telephone,
24               const std::string& p_adresse) :
25       m_nom(p_nom), m_telephone(p_telephone), m_adresse(p_adresse)
26 {
27     PRECONDITION(!p_nom.empty());
28     PRECONDITION(validerTelephone(p_telephone));
29     // m_nom = p_nom;
30     // m_telephone = p_telephone;
31     // m_adresse = p_adresse;
32     POSTCONDITION(m_nom == p_nom);
33     POSTCONDITION(m_telephone == p_telephone);
34     POSTCONDITION(m_adresse == p_adresse);
35     INVARIANTS();
36 }
37
38 std::string Abonne::reqAbonneFormate() const
39 {
40     ostringstream os;
41     // Construction de la string
42     os << "Nom" : " << m_nom << endl;
43     os << "Adresse" : " << m_adresse << endl;
44     os << "Telephone" : " << m_telephone << endl;
45     return os.str();
46 }
47 //*****
48 // Description: L'invariant de cette classe s'assure que les objets Abonne
49 //              sont valides.
50 //*****
51 void Abonne::verifieInvariant() const
52 {
53     INVARIANT(!m_nom.empty());
54     INVARIANT(validerTelephone(m_telephone));
55 }
56
57 //*****
58 // Sommaire: Opérateur de comparaison d'égalité.
59 // Description: On compare l'objet courant avec un objet du même type
60 //              passé en argument en comparant chaque attributs
61 // Entrée: Abonne, la référence d'un objet de la classe Abonne
62 // Sortie: VRAI si la comparaison de chaque attribut est égale.
63 //*****
64 bool Abonne::operator==(const Abonne& p_abonne) const

```

Handwritten annotations on the code:

- A circled '3' is next to the constructor's parameter list.
- A circled '2' is next to the constructor's initialization list.
- A circled '1' is next to the first two PRECONDITION lines.
- A circled '1' is next to the last two POSTCONDITION lines.
- A circled '2' is next to the INVARIANT lines in the verifieInvariant method.
- A circled '1' is next to the INVARIANTS() line in the constructor.
- A circled '1' is next to the INVARIANT(!m_nom.empty()) line.
- A circled '2' is next to the INVARIANT(validerTelephone(m_telephone)); line.

```

1 //*****
2 // Fichier: Entreprise.cpp
3 // Classe: Entreprise
4 //*****
5 #include "Entreprise.h"
6 #include <sstream>
7
8 using namespace util;
9 using namespace std;
10 namespace examenFinal
11 {
12
13 //*****
14 // Description: Constructeur qui initialise la classe avec les donn es fournit
15 //              en argument
16 // Entr e:      nom, nom de l'entreprise
17 //              telephone, le num ro de t l phone de l'entreprise
18 //              adresse, l'adresse sans le code postal
19 //              siteWeb, le site web de l'entreprise
20 //              telecopie, le num ro du t l copieur de l'entreprise
21 //              categorie, indique qu'il s'agit d'un abonn  entreprise
22 //
23 // Note:       Les param tres seront assign s aux attributs de la classe
24 //              seulement si ceux-ci sont valides, sinon un erreur de contrat
25 //              est g n r .
26 //              Voir d finition des attributs pour les conditions pr alables.
27 //*****
28 Entreprise::Entreprise(const std::string& nom, const std::string& telephone,
29                        const std::string& adresse, const std::string& siteWeb,
30                        const std::string& telecopie) :
31                Abonne(nom, telephone, adresse), m_siteWeb(siteWeb), m_telecopie(telecopie) ③
32 {
33     PRECONDITION (validerTelephone(telecopie)); ⑦
34     // m_siteWeb = siteWeb;
35     // m_telecopie = telecopie;
36     POSTCONDITION (m_siteWeb == siteWeb);
37     POSTCONDITION (m_telecopie == telecopie); ①
38     INVARIANTS ();
39 }
40
41 //*****
42 // Description: L'invariant de cette classe s'assure que les objets
43 //              sont valides.
44 //*****
45 void Entreprise::verifieInvariant() const
46 {
47     INVARIANT (validerTelephone(m_telecopie)); ②
48 }
49
50 //*****
51 // Description: Retourne une copie de l'objet courant. Faire une copie allou e
52 //              sur le monceau de l'objet courant.
53 // Sortie:      un objet Entreprise
54 //*****
55 Abonne* Entreprise::clone() const
56 {
57     return new Entreprise(*this); // Appel de l'initialisateur ③
58 }
59 //*****
60 // Description: Retourne les informations de l'abonn  Entreprise selon le
61 //              format d fini suivant:
62 //              abonne Entreprise
63 //              Nom : Presley
64 //              Adresse : 123 rue du King, Ste-Perpetue, Qc, Canada

```

Pour la classe **Entreprise** (14 points)

La classe `Entreprise` est dérivée de la classe `Abonne` telle que présentée plus haut. On vous demande d'écrire les fonctions membre suivantes :

- un constructeur avec paramètres. Les paramètres sont le nom, le numéro de téléphone, l'adresse, le numéro de télécopie et l'adresse du site Web (voir l'interface pour les contraintes sur les attributs).

5

La fonction `verifieInvariant` pour pouvoir implanter le contrat

2

- La méthode `clone` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Abonne` (sans qu'elle soit implémentée dans celle-ci) permet de faire une copie sur le monceau (tas ou heap) de l'objet courant. Ceci permet d'ajouter des Abonnés dans l'annuaire tout en autorisant le polymorphisme (dans la classe `Annuaire`)

3

- La fonction `reqAbonneFormate` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Abonne`. Pensez à faire une déclaration en conséquence. Cette méthode retourne un objet string avec mise en forme ayant le format suivant :

abonne <code>Entreprise</code>	
Nom	: Blo
Adresse	: rue de la medecine
Telephone	: (418) 656-2131
Site Web	: www.joe.com
Telecopie	: 418 656-2134

Pour un abonné Blo, demeurant rue de la medecine, ayant le numéro de téléphone 418 656-2131, ayant le site Web www.joe.com et le numéro de télécopie 418 656-2134 (Pensez à utiliser un objet ostream. La méthode str() appliquée sur un objet ostream retourne un objet string)

4

Pour la classe Annuaire (25 points):

La classe Annuaire comprend deux attributs : un nom et un vector contenant des pointeurs vers des Abonnés (qui peuvent être des objets Entreprise, Residentiel, ...) de l'annuaire. On vous demande d'écrire les fonctions membre suivantes :

- Le constructeur de copie
Annuaire (

5

- Dans un premier temps, on ne veut pas **définir** l'opérateur d'assignation. Cependant, la classe doit rester fiable. Complétez l'interface de la classe pour qu'elle puisse l'être (pensez à une solution simple présentée en cours).
- une méthode `ajouterAbonne`, qui permet d'ajouter des Abonnés à l'annuaire. La méthode `push_back` appliquée sur un `vector` permet d'ajouter un élément passé en paramètre à la fin du `vector` seulement si cet abonné n'est pas déjà présent dans celui-ci. La méthode utilitaire `AbonneEstDejaPresent` permet de le vérifier (à implémenter ci-dessous). Elle prend un Abonné en paramètre. Dans le cas où l'annuaire contient déjà cet abonné, une exception du type `AbonneDejaPresentException` sera lancée. La classe Annuaire est responsable de la gestion de la mémoire.

```

65 // Telephone : (418) 999-9999
66 // Site Web : www.the.king.ca
67 // Telecopie : (418) 999-2222
68 // Sortie: Une chaîne formatte
69 //*****
70 std::string Entreprise::reqAbonneFormate() const
71 {
72     ostringstream os;
73     // Construction de la string
74     os << "abonne Entreprise" << endl;
75     os << Abonne::reqAbonneFormate(); ②
76     os << "Site Web : " << reqSiteWeb() << endl;
77     os << "Telecopie : " << reqTelecopie() << endl;
78
79     return os.str();
80 }
81
82 //*****
83 // Description: Cette méthode constante retourne le site web de l'entreprise
84 // Sortie: std::string : le site Web de l'entreprise
85 //*****
86 const std::string& Entreprise::reqSiteWeb() const
87 {
88     return m_siteWeb;
89 }
90
91 //*****
92 // Description: Cette méthode constante retourne le numéro du télécopieur
93 // Sortie: std::string : le numéro du télécopieur de l'entreprise
94 //*****
95 const std::string& Entreprise::reqTelecopie() const
96 {
97     return m_telecopie;
98 }
99
100 } // namespace examenFinal
101

```

```

1 //*****
2 // Fichier:      Annuaire.cpp
3 // Classe:       Annuaire
4 // Description:  Cette classe repr sente l'annuaire t l phonique d'une ville.
5 // Attributs:    string m_ville : nom de l'annuaire
6 //              vector<Abonne*> m_vAbonnes conteneur des abonn s
7 // Note:         La classe Annuaire est responsable de la gestion de la m moire
8 //*****
9
10 #include <sstream>
11 #include <iostream>
12 #include "Annuaire.h"
13
14 using namespace std;
15 using namespace util;
16 namespace examenFinal
17 {
18
19 //*****
20 // Description:  Constructeur de la classe. On construit un objet
21 //              Annuaire   partir de valeurs pass es en param tre.
22 //
23 //              La classe assigne les attributs seulement si l'abonn 
24 //              est consid r  valide. Autrement, une erreur d'assertion sera
25 //              g n r e.
26 // Entr e:      const std::string& p_ville          : la ville
27 //*****
28 Annuaire::Annuaire(const std::string& p_ville) :
29     m_ville(p_ville)
30 {
31     PRECONDITION(!p_ville.empty());
32
33     POSTCONDITION(m_ville == p_ville);
34     INVARIANTS();
35 }
36
37 Annuaire::Annuaire(const Annuaire& p_annuaire) :
38     m_ville(p_annuaire.m_ville)
39 {
40     for (unsigned i = 0; i < p_annuaire.m_vAbonnes.size(); ++i)  (2)
41     {
42         ajouterAbonne(*(p_annuaire.m_vAbonnes[i]));  (3)
43     }
44     //solution 2
45     vector<Abonne*>::const_iterator iter;
46
47     for (iter = m_vAbonnes.begin(); iter < m_vAbonnes.end(); iter++) 2
48     {
49         ajouterAbonne (**iter); 3
50     }
51 }
52 //*****
53 // Description:  S'il n'est pas d j  pr sent, ajoute un Abonne   m_vAbonnes
54 // Entr e:      const Abonne& nouveauAbonne le nouveau Abonne   ajouter.
55 // Sortie:      rien
56 // Notes:       Si l'abonn  est d j  pr sent, une exeption est g n r e
57 //*****
58
59 void Annuaire::ajouterAbonne(const Abonne& p_nouvelAbonne)
60 {
61     if (abonneEstDejaPresent(p_nouvelAbonne))  (1)
62     {
63         throw AbonneDejaPresentException("cet abonne existe deja");  (2)
64     }

```



```

65     m_vAbonnes.push_back(p_nouvelAbonne.clone());
66
67     POSTCONDITION(!m_vAbonnes.empty());
68 }
69
70 //*****
71 // Description:  Parcours le contenu de l'annuaire dans la liste des abonn s
72 //              et v rifie que le num ro de t l phone n'est pas d j  pr sent
73 //              dans l'annuaire
74 // Sortie:      string noTel: le num ro de t l phone du nouvel abonn 
75 //*****
76
77 bool Annuaire::abonneEstDejaPresent(const Abonne& p_abonne) const
78 {
79     bool present = false;
80
81     //solution 1
82     vector<Abonne*>::const_iterator iter;
83     Abonne* ptrAbonne;
84
85     for (iter = m_vAbonnes.begin(); iter < m_vAbonnes.end(); iter++)
86     {
87         ptrAbonne = *iter;
88         if (*ptrAbonne == p_abonne)
89             present = true;
90     }
91
92     //solution 2
93     vector<Abonne*>::const_iterator iter;
94
95     for (iter = m_vAbonnes.begin(); iter < m_vAbonnes.end(); iter++)
96     {
97         if (**iter == p_abonne)
98             present = true;
99     }
100
101     //solution 3
102     vector<Abonne*>::const_iterator iterI = m_vAbonnes.begin();
103     vector<Abonne*>::const_iterator finI = m_vAbonnes.end();
104
105     while (iterI != finI)
106     {
107         ptrAbonne = *iterI;
108         if (*ptrAbonne == p_abonne)
109         {
110             present = true;
111             break;
112         }
113         iterI++;
114     }
115
116     return present;
117 }
118
119 //*****
120 // Description:  Imprime Le nom de la ville et pour chaque abonn 
121 //              le nom de l'abonn , son adresse, son le num ro de t l phone
122 //              sa cat gorie ainsi que les informations propres   sa cat gorie
123 // Sortie:      string: contient les informations   afficher
124 //*****
125 string Annuaire::reqAnnuaireFormate() const
126 {
127     ostream os;
128
129     os << "Annuaire de : " << m_ville << endl << "=====

```

```

129         << endl;
130     //solution 1
131     // for (unsigned i = 0; i < m_vAbonnes.size(); ++i)
132     // {
133     //     os << m_vAbonnes[i]->reqAbonneFormate();
134     //     os << endl << "-----" << endl;
135     // }
136     //solution 2
137     vector<Abonne*>::const_iterator iterI = m_vAbonnes.begin();
138     vector<Abonne*>::const_iterator finI = m_vAbonnes.end();
139     while (iterI != finI)
140     {
141         os << (*iterI)->reqAbonneFormate();
142         os << endl << "-----" << endl;
143         iterI++;
144     }
145
146     return os.str();
147 }
148 //*****
149 // Description: Permet d'envoyer l'objet dans un ostream en écrivant en premier
150 // le nombre d'abonnés qu'il y a dans le vecteur et ensuite pour
151 // chacun des abonnés, la catégorie sur une ligne et les infos
152 // sur l'abonné, chacune étant sur une ligne
153 // Entrée: std::ostream& os, le flux contenant l'objet
154 // Annuaire& obj, l'objet à inclure dans le flux.
155 // Sortie: une référence sur le flux
156 //*****
157 std::ostream& operator<<(std::ostream& p_os, const Annuaire& p_annuaire)
158 {
159     p_os << p_annuaire.reqAnnuaireFormate();
160     return p_os;
161 }
162
163 //*****
164 // Description: Destructeur de la classe.
165 // Note: Il faut désallouer tous les objets abonne
166 //*****
167 Annuaire::~Annuaire()
168 {
169     // supprime chaque abonné pointé
170     for (unsigned i = 0; i < m_vAbonnes.size(); i++)
171     {
172         delete m_vAbonnes[i];
173     }
174 }
175
176 //*****
177 // Description: L'invariant de cette classe s'assure que de l'annuaire
178 // est valide. Voici les détails :
179 // - m_codePostal doit avoir un format valide
180 //*****
181 void Annuaire::verifieInvariant() const
182 {
183     INVARIANT(!m_ville.empty());
184 }
185
186 } // namespace examenFinal
187

```

(2)

(2)

) + (2) si J'enlève

(2)

(2)

ajouterAbonne (

5

- La méthode `abonneEstDejaPresent`

5

- un destructeur. La méthode `size` appliquée sur un vector retourne le nombre d'éléments qu'il contient.

4

- une méthode `reqAnnuaireFormate` qui retourne un objet string formaté (voir le prototype dans l'interface). **Utiliser ici obligatoirement un iterator.** Le format de l'objet string doit être le suivant :

```
Liste des abonnées
Annuaire de : Quebec
=====
abonne Entreprise
Nom          : Blo
Adresse      : rue de la medecine
Telephone    : 418 656-2131
Site Web     : www.joe.com
Telecopie    : 418 656-2134
```

Dans l'exemple, l'annuaire de Quebec compte 1 abonné de catégorie Entreprise, (Pensez à utiliser une ostream)

A red handwritten letter 'c' is centered on a set of horizontal lines. The letter is formed with a single stroke, starting from the middle line, curving down to the bottom line, and then curving back up to the middle line.

Rappel : Avez-vous pensé à mettre à jour toutes les interfaces ?

3. Classe AbonneDejaPresentException (3 points)

Cette classe permet de gérer l'exception de l'ajout d'un doublon d'abonnés dans l'annuaire. Cette classe hérite de `std::runtime_error` parce que cette exception est une erreur qui pourra toujours se produire, ce n'est pas une erreur de programmation.

Elle contient seulement une méthode qui est le constructeur suivant :

```
AbonneDejaPresentException (const std::string& p raison);
```

Ce constructeur ne fait qu'appeler le constructeur de la classe parent en lui passant la raison comme paramètre.

La méthode `what()` de la classe `std::exception` au haut de la hiérarchie permet d'obtenir la string qu'on a passé.

On vous demande de compléter l'interface de cette classe

```
// Fichier AbonneDejaPresentException.h
```

```
#include <stdexcept>
```

```
class AbonneDejaPresentException  
{
```

3

et d'écrire l'implémentation du constructeur.

```
AbonneDejaPresentException (const std::string& p_raison)
```

4. Test unitaire (12 points)

On vous demande d'implémenter le test unitaire du constructeur de la classe Abonne, ceci en utilisant les fonctionnalités offertes par le framework GoogleTest. Attention, n'oubliez pas que la classe Abonne est abstraite.

Les principales macros et assertions sont rappelées en annexe.

```
/**  
 * \file AbonneTesteur.cpp  
 * Implantation des tests unitaires de la classe Abonne  
 */  
  
#include <gtest/gtest.h>  
#include " Abonne.h"  
  
using namespace std;  
using namespace examenFinal;
```

```
1  /*
2  * AbonneDejaPresentException.h
3  */
4
5  #ifndef ABONNEDEJAPRESENTEXCEPTION_H_
6  #define ABONNEDEJAPRESENTEXCEPTION_H_
7
8  #include <stdexcept>
9  #include <string>
10
11 namespace examenFinal
12 {
13 class AbonneDejaPresentException: public std::runtime_error ①
14 {
15 public:
16     AbonneDejaPresentException(const std::string& raison); ②
17 };
18 } // namespace examenFinal
19
20 #endif /* ABONNEDEJAPRESENTEXCEPTION_H_ */
21
22 /*
23 * AbonneDejaPresentException.cpp
24 */
25
26 #include "AbonneDejaPresentException.h"
27 namespace examenFinal {
28
29 AbonneDejaPresentException::AbonneDejaPresentException(
30     const std::string& raison) : ③
31     std::runtime_error(raison)
32 {
33 }
34 } // namespace examenFinal
35
```

12

```

1  /*
2  * AbonneTesteur.cpp
3  * Implantation des tests unitaires de la classe Abonne
4  * Abonne étant une classe abstraite, on crée une classe concrète
5  * dérivée de cette classe afin de pouvoir effectuer les tests
6  *      Author: theud1
7  */
8
9  #include <gtest/gtest.h>
10 #include "Abonne.h"
11 #include "ContratException.h"
12
13 using namespace std;
14 using namespace examenFinal;
15
16 using namespace std;
17 using namespace examenFinal;
18
19 // Création de la classe concrète de Abonne pour les tests
20 class AbonneDeTest: public Abonne
21 {
22 public:
23     AbonneDeTest(const std::string& p_nom, const std::string& p_telephone,
24                 const std::string& p_adresse) :
25         Abonne(p_nom, p_telephone, p_adresse)
26     { };
27     virtual string reqAbonneFormate() const
28     {
29         return Abonne::reqAbonneFormate();
30     }
31     ;
32     virtual Abonne* clone() const
33     {
34         return new AbonneDeTest(*this);
35     }
36     ;
37 };
38
39 //*****
40 // Fixture UnAbonne pour la création d'un objet Abonne
41 // utilisé dans les tests
42 // *****
43 class UnAbonne: public ::testing::Test
44 {
45 public:
46     UnAbonne() :
47         unAbonne("Blo", "(418) 656-2131", "rue de la medecine")
48     { };
49     AbonneDeTest unAbonne;
50 };
51
52 //*****
53 // Test du Constructeur
54 // Cas valide:
55 //     ConstructeurParametresValides : Création d'un objet Abonne et
56 //     vérification de l'assignation de tous les attributs
57 // Cas invalides:
58 //     ConstructeurReferenceInvalide: Création avec non vide
59 //     ConstructeurTelephoneInvalide: vérifi avec validerTelephone
60 //*****
61 TEST(UneAbonneConstructeur, ConstructeurParametresValides)
62 {
63     AbonneDeTest unAbonne("Blo", "(418) 656-2131", "rue de la medecine");
64     ASSERT_EQ("Blo", unAbonne.reqNom());

```



```
65     ASSERT_EQ("(418) 656-2131", unAbonne.reqTelephone());
66     ASSERT_EQ("rue de la medecine", unAbonne.reqAdresse());
67
68 }
69
70 // Cas invalides:
71 // ConstructeurReferenceInvalide: Cr ation avec nom vide
72 TEST(UneAbonneConstructeur, ConstructeurReferenceInvalide)
73 {
74     ASSERT_THROW(AbonneDeTest("", "(418) 656-2131", "rue de la medecine"),
75                  PreconditionException);
76 // ConstructeurTelephoneInvalide: t l phone format non valide
77 TEST(UneAbonneConstructeur, ConstructeurTelephoneInvalide)
78 {
79     ASSERT_THROW(AbonneDeTest("Blo", "418 656 2131", "rue de la medecine"),
80                  PreconditionException);
81 }
82 //*****
83 // Test de la m thode d'acc s string Abonne::reqAbonneFormate() const
84 // Cas valide:
85 // reqAbonneFormate : V rification du retour de la m thode
86 // Cas invalide:
87 // Aucun d'identifi 
88 //*****
89 TEST_F(UnAbonne, reqAbonneFormate)
90 {
91     cout << unAbonne.reqAbonneFormate();
92 // ASSERT_EQ("liq 256\n50\n", unAbonne.reqAbonneFormate());
93 }
94
```

Annexe

`ostringstream` fournit une interface pour manipuler des chaînes de caractères comme des flux de sortie.

membres public :

`string str ( ) const;` pour obtenir l'objet `string` associé (fonction membre publique)

La méthode `size()` appliquée sur un `vector` retourne le nombre d'éléments qu'il contient.

La méthode `begin()` appliquée sur un `vector` retourne un `iterator` pointant sur le début de celui-ci.

La méthode `end()` appliquée sur un `vector` retourne un `iterator` pointant un élément passé la fin de celui-ci.

Balises Doxygen

```
///! ... Documentation ...
\file
Permet de créer un bloc de documentation pour un fichier source ou d'en-tête.
\brief
Permet de créer une courte description dans un bloc de documentation. La description peut se
faire sur une seule ligne ou plusieurs.
\author
Permet d'ajouter un nom d'auteur (ex: l'auteur d'un fichier ou d'une fonction).
\version
Permet l'ajout du numéro de version (ex: dans le bloc de documentation d'un fichier).
\date
Permet l'ajout d'une date.
\struct
Permet la création d'un bloc de documentation pour une structure.
\enum
Permet la création d'un bloc de documentation pour une énumération de constantes.
\fn
Permet la création d'un bloc de documentation pour une fonction ou méthode.
\def
Permet la création d'un bloc de documentation pour une macro ou constante symbolique.
\param
Permet d'ajouter un paramètre dans un bloc de documentation d'une fonction ou d'une macro.
Cette balise permet aussi d'ajouter en option un tag pour montrer qu'il s'agit d'un argument
entrant, sortant ou les deux en précisant au choix: [in], [out]
\return
Permet de décrire le retour d'une fonction ou d'une méthode.
\bug
Permet de commencer un paragraphe décrivant une bogue (ou plusieurs).
\class
Permet de créer un bloc de documentation d'une classe (C++). L'utilisation est identique à
la balise \struct
\namespace
Permet de créer un bloc de documentation pour un espace de nom (C++).
```

Framework GoogleTest :

Assertions Basiques

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_TRUE(condition);</code>	<code>EXPECT_TRUE(condition);</code>	<i>condition</i> is true
<code>ASSERT_FALSE(condition);</code>	<code>EXPECT_FALSE(condition);</code>	<i>condition</i> is false

Comparaisons binaires (comparaisons entre deux valeurs v1 et v2 (valeurs comparables))

Fatal assertion	Nonfatal assertion	Verifies
ASSERT_EQ(<i>expected</i> , <i>actual</i>);	EXPECT_EQ(<i>expected</i> , <i>actual</i>);	<i>expected</i> == <i>actual</i>
ASSERT_NE(<i>val1</i> , <i>val2</i>);	EXPECT_NE(<i>val1</i> , <i>val2</i>);	<i>val1</i> != <i>val2</i>
ASSERT_LT(<i>val1</i> , <i>val2</i>);	EXPECT_LT(<i>val1</i> , <i>val2</i>);	<i>val1</i> < <i>val2</i>
ASSERT_LE(<i>val1</i> , <i>val2</i>);	EXPECT_LE(<i>val1</i> , <i>val2</i>);	<i>val1</i> <= <i>val2</i>
ASSERT_GT(<i>val1</i> , <i>val2</i>);	EXPECT_GT(<i>val1</i> , <i>val2</i>);	<i>val1</i> > <i>val2</i>
ASSERT_GE(<i>val1</i> , <i>val2</i>);	EXPECT_GE(<i>val1</i> , <i>val2</i>);	<i>val1</i> >= <i>val2</i>

Assertions d'exceptions

Fatal assertion	Nonfatal assertion	Verifies
ASSERT_THROW(<i>statement</i> , <i>exception_type</i>);	EXPECT_THROW(<i>statement</i> , <i>exception_type</i>);	<i>statement</i> throws an exception of the given type
ASSERT_ANY_THROW(<i>statement</i>);	EXPECT_ANY_THROW(<i>statement</i>);	<i>statement</i> throws an exception of any type
ASSERT_NO_THROW(<i>statement</i>);	EXPECT_NO_THROW(<i>statement</i>);	<i>statement</i> doesn't throw any exception

Test simple:

```
TEST(nom_cas_test, nom_test)
```

```
{
```

```
    ... corps de test ...
```

```
}
```

Tests utilisant un fixture, utiliser TEST_F()

Structure d'un fixture :

```
class FooTest : public ::testing::Test {
```

```
//constructeur
```

```
...
```

```
//méthodes
```

```
...
```

```
//attributs utilisés dans les tests, initialisés dans le constructeur
```

```
...
```

```
};
```